

Driving Product Improvement Through Experimentation: A/B Testing Contextual Term Suggestions in NokiDoki

-Rithvik Rangaraj

Problem Statement

At Nokia, technical writers are responsible for producing precise and standardized documentation across various telecom domains. Even minor inconsistencies such as a misplaced hyphen, incorrect casing, or outdated terminology, can lead to significant downstream consequences in telecom protocols, configuration scripts, or customer-facing material.

NokiDoki was introduced in 2019 as an internal term suggestion tool to help enforce consistency across documentation. However, it relied on a static, rule-based pattern-matching engine that failed to adapt to evolving standards and language. Over time, it began surfacing obsolete or irrelevant terms, frustrating writers and increasing the likelihood of non-compliant or error-prone documentation.

This not only undermined the reliability of internal manuals but also introduced risk into downstream engineering and operational processes. The lack of contextual awareness, personalization, and semantic understanding created a significant gap between user expectations and tool performance. Many users opted to maintain a personal dictionary of terms they felt would be reused in their documentations.

User Pain Points

NokiDoki was originally built to help technical writers maintain consistency in documentation by suggesting standardized terminology. However, over time, its rigid rule-based backend failed to keep pace with evolving nomenclature, leading to growing dissatisfaction and workarounds. Key pain points included:

- **Obsolete Suggestions:** The system continued to recommend outdated terms based on legacy usage frequency, leading to confusion and non-compliant documentation.
- **Irrelevant Results:** Without contextual understanding, the tool often surfaced terms that were syntactically similar but semantically incorrect, increasing the likelihood of serious technical misinterpretation.
- **Loss of Trust:** Writers frequently ignored NokiDoki altogether, citing low relevance and wasted time. Many resorted to manually searching internal repositories or old manuals to find accurate terms.
- **Workflow Friction:** Lack of personalization or session awareness meant users had to rewrite queries, manually compare options, or ask colleagues, all of which slowed down the authoring process.
- **Shadow Systems:** In frustration, many writers began maintaining their own “personal dictionaries” of preferred terms, stored in spreadsheets or sticky notes. This fragmented standardization further creating parallel, unverified sources of truth across teams.

My Role

As the analytics and experimentation lead on the NokiDoki initiative, I was responsible for designing and executing the framework to evaluate a new AI-driven recommendation system. My focus was to ensure that product decisions were rooted in data and optimized for actual user behavior.

Key responsibilities:

- Defined the product success metrics in collaboration with PMs and documentation leads, aligning them with business goals (speed, accuracy, engagement).
- Designed the A/B test structure, including variant logic, session-level randomization strategy, and backend flag integration.
- Instrumented the user funnel across key touchpoints, from tool launch to search queries, term selection, and post-action feedback.
- Built interactive dashboards for real-time monitoring of experiment KPIs (Power BI + SQL), enabling fast iteration cycles.
- Analyzed test results, synthesized quantitative outcomes with qualitative writer feedback, and presented actionable insights to product and engineering teams.
- Advocated for a hybrid approach after identifying diverging preferences among writers (global vs. context-aware), proposing a future toggle feature to support both workflows.
- Throughout the project, I worked closely with engineering, UX, and product teams to ensure our experiments were robust, actionable, and aligned with real user needs.

Solution Overview

To address the limitations of NokiDoki's static rule-based engine, we implemented a dynamic collaborative filtering-based recommendation system that could adapt to evolving usage patterns and individual writing contexts. The new system analyzed prior term selections, editor behavior, and recent session activity to suggest more relevant and up-to-date terminology. Unlike the legacy engine that surfaced high-frequency terms globally, this model prioritized contextual relevance, helping users find the right term faster and with higher confidence.

Key solution components:

- **Collaborative Filtering Model:** Trained on historical term selection and session logs to identify patterns in term co-occurrence and user preference.
- **Contextual Signal Layer:** Enhanced the recommendation engine by incorporating session-level metadata, such as department, document type, and previously used terms.
- **A/B Test Deployment:** Delivered two variants:
 1. **Global Ranking (Control):** Based on term popularity across all users
 2. **Context-Aware Ranking (Test):** Personalized suggestions using recent session context
- **Frontend Integration:** Suggestions displayed as a ranked Top-N list below the search bar. Drag-and-drop UX remained unchanged to reduce learning friction.
- **Feedback Widget:** Light-touch in-app rating system (1–5 scale) to measure user satisfaction with suggestions. This modular architecture enabled us to experiment, measure, and iterate without disrupting core writer workflows, and paved the way for future personalization enhancements.

Experiment Setup

To evaluate the impact of the new recommendation engine, we conducted a controlled A/B experiment across active NokiDoki users. The test was designed to isolate the effect of context-aware term ranking on search efficiency, suggestion relevance, and overall tool engagement.

Test Design

- **Population:** All active technical writers using NokiDoki over a 4-week testing window.

- **Randomization Unit:** Session-level randomization using a hashed session ID. This ensured a writer would consistently receive one variant per session, avoiding cross-variant contamination.

Variant Groups:

1. **Group A, Control:** Received global canonical suggestions, ranked by overall usage frequency across all users.
2. **Group B, Test:** Received context-aware suggestions, ranked based on recent session context (e.g., prior documents accessed, team, or domain).

Flagging Logic: Session flags were applied server-side and passed through API response headers. The frontend dynamically rendered suggestions based on the assigned variant.

Blind Testing: Users were unaware of which group they were assigned to, reducing bias in behavior or feedback.

Funnel Instrumentation

We instrumented each major user interaction within NokiDoki to capture quantitative data along the writer’s journey. Events were logged as follows:

Funnel Step	Event Logged
Tool Launch	nokidoki_start
Term Search	search_query
Suggestions Rendered	suggestions_rendered
Term Imported to Editor	term_imported
Feedback Submitted (1–5)	suggestionRated
Search Rewritten/Modified	search_rewrite
Session Exit	nokidoki_exit

Bias & Integrity Controls

To ensure the experiment was clean and reliable:

- Sessions were evenly sampled across times of day and weekdays to reduce temporal bias.
- Feature parity between both variants was confirmed pre-rollout through QA and shadow deployments.
- Missing or null events were actively monitored during the experiment to ensure data quality.
- Metrics were normalized across writer tenure and department to account for skill or domain-based differences.
- This rigorous setup ensured we could confidently attribute observed changes in user behavior to the recommendation engine, not to external noise or unrelated system changes.

Metrics Tracked

1. Search Latency

Definition: Time (in seconds) between the search_query event and the term_imported event.

Why it matters: Measures the efficiency of the search process. Lower latency implies that users are finding the right terms faster, reducing writing friction.

2. Term Adoption Rate

Definition: Ratio of term_imported events to suggestions_rendered events within a session.

Why it matters: Indicates how often users find suggestions helpful. Higher adoption reflects better relevance and usability of recommended terms.

3. Relevance Rating

Definition: Average user rating submitted via the suggestion_rated event (scale: 1 to 5).

Why it matters: Captures subjective user satisfaction with the suggestion quality. Higher scores indicate increased trust in the tool.

4. Zero-Click Sessions

Definition: Percentage of sessions where no term was imported, despite suggestions being shown.

Why it matters: A leading indicator of poor recommendation relevance. Lower zero-click rates reflect improved suggestion utility.

5. Query Rewrite Rate

Definition: Percentage of sessions where users modified their search query at least once (i.e., multiple search_query events).

Why it matters: Reflects user frustration or confusion. Fewer rewrites suggest the system is returning useful results on the first try.

6. Ignore Rate

Definition: Proportion of Top-N suggestions that were never clicked or imported, relative to total rendered suggestions.

Why it matters: Indicates noise in the recommendation list. Lower ignore rates signal higher precision in suggestion ranking.

7. Repeat Usage Rate

Definition: Percentage of unique users who returned to use NokiDoki again within 7 days.

Why it matters: A proxy for long-term user value. Higher repeat usage implies increased trust and integration into daily workflows.

Analytics Dashboards & Visualization

- **KPI Overview Cards:** Displayed average search time, adoption rate, and 7-day return rate by variant
- **Bar & Line Charts:** Tracked daily engagement trends and performance deltas between control and test groups
- **Heatmaps:** Visualized session behavior across document types, departments, and term categories

- **Boxplots & Histograms:** Showed distribution of user ratings and term selection time (to detect outliers)
- **Sankey Diagram:** Mapped user journeys from search to import vs. drop-off points
- **Term-Level Drilldowns:** Allowed product teams to filter by term, view click-through rates, ignore rates, and low-rated entries

Use Cases & Outcomes

- **Weekly Experiment Reviews:** Dashboards were used to brief stakeholders and adjust rollout tactics mid-test if needed.
- **Leadership Buy-in:** Summary views were adapted for slide decks presented to Documentation and Engineering leadership, helping secure sign-off for full rollout.
- **Backlog Prioritization:** Drilldowns helped identify commonly ignored or low-rated terms, leading to targeted improvements in the ranking algorithm and terminology database.

Results Summary

The experiment ran successfully over a 4-week window with statistically significant data across both test and control groups. The context-aware recommendation system (Test Group) showed consistent and measurable improvements across all key metrics compared to the legacy global ranking (Control Group).

Quantitative Outcomes

Metric	Control (Global Ranking)	Test (Context-Aware)	Delta
Search Latency	7.4 seconds	4.6 seconds	↓ 2.8 seconds
Term Adoption Rate	41%	56%	↑ 15 percentage pts
Relevance Rating	3.2 / 5	4.1 / 5	↑ 0.9
Zero-Click Sessions	29%	21%	↓ 25%
Ignore Rate	48%	39%	↓ 18%
Repeat Usage (7-day)	52%	64%	↑ 12 percentage pts

All observed changes were statistically significant at $p < 0.01$.

- Writers in the context-aware group reported that suggestions “felt more intuitive” and “required less rework or guesswork.”
- Several users mentioned that they “stopped using their personal dictionaries” during the test period, a strong signal of restored tool trust.
- Feedback scores indicated higher satisfaction in specialized domains (e.g., 5G, OSS) where contextual nuance matters more.
- The new engine successfully shifted writer behavior from manual search workarounds to trusting in-tool recommendations.
- Faster task completion, fewer query rewrites, and higher engagement indicate not just usability wins but long-term trust recovery.

Results were shared with Documentation and Knowledge Management leadership, who approved:

- Full rollout of the context-aware model
- Exploration of a toggle system to support both global and personalized modes
- A future roadmap for semantic tagging and LLM-based expansion

Bias & Integrity Controls

1. Randomization & Test Isolation

- Session-Level Randomization: Each user session was randomly assigned to either the global or context-aware variant using a hashed session ID, ensuring stable exposure within a session and preventing crossover contamination.
- Server-Side Flag Control: Recommendation logic was determined entirely backend-side, with no influence from client settings or cookies.
- Blind Exposure: Writers were not informed of which variant they were using, minimizing expectancy bias or self-reported influence.

2. Temporal Controls

- Sessions were uniformly distributed across weekdays and time slots to eliminate bias from time-of-day or week-specific behaviors.
- The 4-week testing window ensured inclusion of both peak and off-peak usage patterns.

3. Data Integrity Measures

- All funnel events (search_query, term_imported, suggestion_rated, etc.) were schema-validated and QA-tested prior to launch.
- Real-time dashboards flagged any unexpected event drop-offs, duplicate entries, or null values, all of which remained below acceptable thresholds throughout the test period.

4. Statistical Checks

- Final results underwent significance testing using two-tailed t-tests and proportion Z-tests, with confidence thresholds set at 95%+ ($p < 0.01$).
- Normalization was applied to adjust for departmental size, region, and writer tenure, ensuring that outcomes were not driven by subgroup anomalies.

5. Confounder Mitigation

- Confirmed identical UI/UX across both variants (other than the ranking logic) to isolate the impact of the recommendation system.
- Monitored for feature rollouts, outages, or concurrent experiments during the test window, none were active, confirming a clean test environment.

Edge Cases & Failure Modes

Synonym & Ambiguity Handling

- Certain technical terms (e.g., “5GC” vs “5G Core”, “eNB” vs “e-NodeB”) appeared frequently in user feedback due to ambiguous meanings or overlapping usage.
- The collaborative filtering model, which relied on co-occurrence patterns, sometimes failed to distinguish contextually distinct but statistically similar terms.

Solution: Added a “disambiguation warning” tag in internal testing and flagged these cases for future rule layering or semantic modeling.

Cold Start for New Terms

- Newly introduced or niche terms had insufficient historical usage data, making it hard for the recommender to rank them accurately.
- In some cases, relevant but rare terms were buried below the fold or omitted from Top-N entirely.

Solution: Introduced a minimal baseline relevance score for all recently introduced terms to ensure visibility during the model's learning period.

Feedback Loop Gaps

- Although user ratings (1–5) were collected, the feedback was not yet integrated into model retraining due to infrastructure limitations.

Result: Suggestions with consistent low ratings were still occasionally shown, reducing trust in edge cases.

Solution: Logged all low-rated suggestions into a review dashboard for manual override; proposed future integration with an active learning pipeline.

Zero-Result Queries

- Some queries yielded no suggestions due to unseen phrasing (e.g., acronym variants, hyphenation errors).
- These incidents accounted for ~3% of all sessions.

Solution: Logged these queries and proposed fuzzy matching or token normalization in the next model iteration.

Power Users Bias

- Some senior writers with heavy usage patterns had disproportionate influence on collaborative filtering logic, creating popularity bias in certain term categories.

Solution: Temporarily capped user weight in model training to prevent over-personalization based on a small sample.

Limitations of the Current System

Despite measurable improvements, the current system had inherent constraints tied to model design, tooling, and infrastructure maturity:

- **Cold Start Problem:** Collaborative filtering required historical interaction data. Newly introduced terms or first-time writers lacked sufficient signals for effective personalization.
- **Lack of Semantic Understanding:** Recommendations were based on statistical co-occurrence, not semantic similarity. This occasionally resulted in syntactically similar but contextually incorrect suggestions.
- **No Active Feedback Loop:** Although users rated term suggestions, these scores were not yet integrated into model retraining or re-ranking logic.
- **Static Context Windows:** Context-awareness was session-based but didn't extend across longer time frames or user-specific history beyond the current session.
- **Infrastructure Trade-offs:** Due to internal tool constraints, more advanced NLP techniques or hybrid models (e.g., TF-IDF + CF) were deferred to future cycles.

Future Scope & Roadmap Opportunities

To further evolve the NokiDoki recommendation system and scale its impact across Nokia's documentation ecosystem, the following initiatives were identified:

Introduce a Hybrid Toggle Option

- Allow writers to toggle between global canonical and context-aware modes, catering to different use cases (e.g., consistency vs. personalization).

Integrate LLM-Based Semantic Retrieval

- Leverage transformer models or domain-specific embeddings to suggest terms based on meaning, not just usage history. This would improve handling of ambiguous or compound technical phrases.

Implement Feedback-Driven Learning Loops

- Incorporate user rating data directly into model tuning or confidence scoring, improving responsiveness to low-rated suggestions over time.

Term Taxonomy & Topic Tagging

- Tag terms by category (protocol, product, hardware type) and allow writers to filter suggestions, improving control and reducing cognitive load.

Persistent User Profiles

- Move beyond session-based context by introducing lightweight user histories, allowing the tool to adapt based on long-term behavior and department-level preferences.

Broader Deployment Across Tools

- Extend the recommendation engine into related documentation environments (e.g., Confluence plugins, internal IDEs) to unify terminology guidance.

Strategic Impact

Organizational Benefits

Improved Documentation Quality

- By reducing inconsistent terminology and obsolete references, the tool directly improved the clarity, compliance, and reusability of technical manuals across product lines, decreasing the risk of downstream miscommunication in engineering, QA, and customer support.

Time Savings at Scale

- With an average reduction of ~2.8 seconds per term lookup and hundreds of searches performed daily, the new model delivered a monthly productivity gain equivalent to 30+ hours of technical writer time saved, enough to shift critical resources to higher-impact writing tasks.

Reduced Shadow Systems

- The elimination of personal dictionaries and workaround workflows signaled renewed trust in NokiDoki as a source of truth, promoting organizational consistency and reducing duplicated knowledge maintenance efforts.

Reusable Analytics Infrastructure

- The funnel instrumentation, dashboards, and A/B test design patterns served as templates for experimentation in other internal tools enabling scalable analytics practices across the documentation and developer experience teams.

Strategic Alignment

Supports Nokia's Knowledge Management Roadmap

- Aligned with the long-term initiative to modernize internal documentation tools with AI, analytics, and semantic enrichment.

Strengthens Cross-Team Efficiency

- Reduced terminology inconsistencies improved communication between documentation, engineering, and product teams, particularly in fast-paced or regulatory-heavy domains like 5G, OSS, and Network Infrastructure.

Establishes a Data-Driven Product Culture

- Demonstrated how small internal tools can be tested, measured, and improved through experimentation, creating a blueprint for agile product thinking within traditionally static enterprise environments.

Cross-Functional Collaboration

Key Collaborators and Contributions

Product Management

- Co-defined success metrics, product hypotheses, and A/B test goals
- Collaborated on decision-making post-experiment to recommend rollout strategy

Engineering Team

- Implemented backend flagging logic, API routing, and event instrumentation
- Ensured consistent and reliable variant delivery for the test group

Machine Learning Engineering (MLE) Team

- Developed and fine-tuned the collaborative filtering engine
- Integrated contextual signals and ensured model outputs aligned with user session data
- Provided model explainability hooks and debug tools during test rollout

UX & Design

- Advised on suggestion list layout and feedback capture UX
- Helped preserve a non-intrusive user experience across both variants

Technical Writers (End Users)

- Shared firsthand feedback on term relevance and contextual usefulness
- Participated in pilot sessions and completed in-app satisfaction ratings

Documentation Leadership

- Reviewed weekly dashboards and championed the rollout based on data
- Aligned this initiative with broader knowledge governance and tooling strategy

Key Learnings

- **Context is everything:** Relevance improves dramatically when user behavior, recent activity, and domain specialization are considered, even in traditionally static tools like documentation platforms.
- **Data wins arguments:** The experiment's success demonstrated that rigorous A/B testing and clear metrics can align cross-functional teams and accelerate product decision-making.
- **Small changes compound:** A faster, more relevant term suggestion might save only a few seconds per search, but scaled across hundreds of users and thousands of sessions, it delivers real operational leverage.
- **Trust is earned, not assumed:** Many writers had abandoned the legacy tool. Rebuilding trust required not only better suggestions, but transparent measurement and responsive iteration.